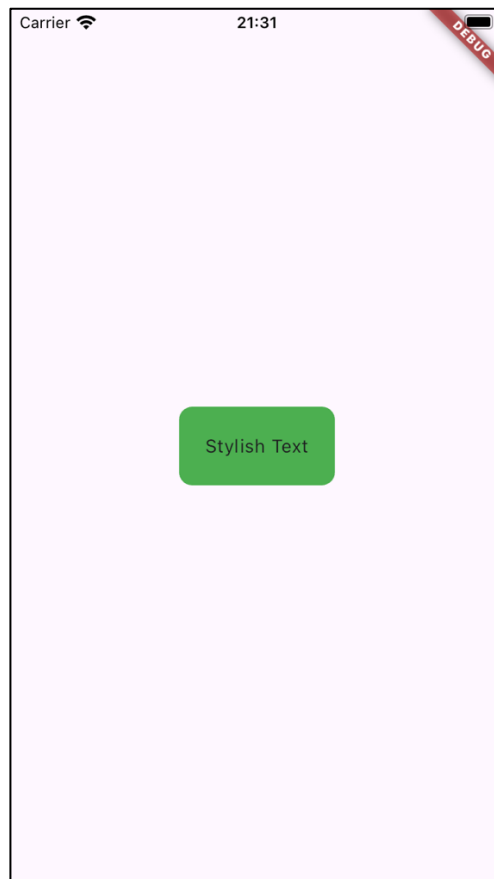




### Why Does Flutter Use Functions and Properties Inside Widgets?

- a. Why Are Functions and Properties Used in Widgets?
  - Flutter allows functions and properties to be passed inside widgets.
  - This helps define both appearance (UI) and behavior (logic) in one place.
  - Instead of writing UI and logic separately (like older UI frameworks), Flutter keeps them together.
- b. Example: Adding Actions with onPressed (Button Example)
  - You can pass a function inside the onPressed property of a button widget.
  - This function tells the button what to do when clicked.
  - Both appearance (UI) and behavior (logic) stay together in one place.
  - This is example code :

```
ElevatedButton(  
  onPressed: () {  
    print('Button clicked!'); // This function runs when  
the button is pressed  
  },  
  child: Text('Click Me'), // The text shown inside the  
button  
);
```

- Explanation:
  - onPressed is a property where a function is passed.
  - The function executes when the button is clicked.

- The UI (Text('Click Me')) and logic (onPressed) are together.
- ◇ Separating Functions for Better Code Organization
  - Instead of defining functions inside widgets, you can define them separately and reference them.
  - This makes the code cleaner and easier to maintain.
  - Useful when the function is complex or used multiple times.
  - For example, this is code using separate function :

```
void buttonClicked() {
    print('Button clicked!'); // This function runs when
    the button is pressed
}

ElevatedButton(
    onPressed: buttonClicked, // Just reference the
    function by its name
    child: Text('Click Me'), // The text shown inside the
    button
);
```

- Benefits of Separating Functions:
  - ✓ *Reusability* → The same function can be used in multiple places.
  - ✓ *Clarity* → UI code remains clean and focused on design, while logic is handled separately.
  - ✓ *Maintainability* → If the button behavior changes, update only the function, not every button.
- c. Example: Using Properties for Styling (TextStyle in Text Widget)
  - Properties allow you to customize a widget's appearance.
  - Instead of defining styles separately, you can pass a TextStyle object inside the style property.
  - This makes customization more flexible and readable.
  - This is example code :

```
Text(
    'Hello, Flutter!',
    style: TextStyle( // TextStyle customizes the appearance
    of the text
        fontSize: 24, // Makes the text larger
        fontWeight: FontWeight.bold, // Makes the text bold
        color: Colors.blue, // Changes the text color to blue
    ),
);
```

Hello, Flutter!

- Explanation:
  - style is a property that takes a TextStyle object.

- `fontSize`, `fontWeight`, and `color` are keys that define text appearance.
- Customization is done directly inside the widget instead of a separate stylesheet.

## **Summary of Fundamental Concepts**

- Key-value pairs in Flutter allow customization and flexibility.
- They override default values to modify UI appearance & behavior.
- Flutter's named parameters improve readability and maintainability.
- Flutter's key-value structure resembles JSON but serves different purposes.
  
- Nesting widgets allows structuring UI elements clearly.
- Keeps related components together, making code more readable.
- Customization is easier by wrapping widgets inside style-defining widgets.
- Flutter UI is entirely built using a nested widget tree.
  
- Functions inside widgets define behavior (e.g., `onPressed`).
- Separating functions improves reusability, clarity, and maintainability.
- Properties inside widgets customize appearance (e.g., `style` in `Text`).
- Flutter combines UI and logic in one place, making code easier to manage.

- When you need a consistent top bar for your app.
- When you want to add a title or navigation buttons.
- When you need search, settings, or more options icons.
- When you want to add tabs or extra content below the AppBar.

## Text Widget

- What is the Text Widget?
  - Displays text on the screen in a Flutter app.
  - Supports both plain and styled text.
  - Can be used for static messages, dynamic updates, and responsive design.
- Key Properties of the Text Widget
  - Commonly used properties:

| Property  | Description                             | Example                                     |
|-----------|-----------------------------------------|---------------------------------------------|
| data      | The text to display (required).         | 'Hello, Flutter!'                           |
| style     | Customizes the text appearance.         | TextStyle(fontSize: 18, color: Colors.blue) |
| textAlign | Aligns text (left, center, right).      | TextAlign.center                            |
| maxLines  | Limits the number of text lines.        | maxLines: 2                                 |
| overflow  | Handles text overflow (ellipsis, fade). | TextOverflow.ellipsis                       |
| softWrap  | Controls text wrapping.                 | softWrap: true                              |

- Example: Styling Text in Flutter
  - Code Example:

```
Text (
  'Welcome to Flutter!', // The text to display
  style: TextStyle(
    fontSize: 20,           // Sets the font size
    color: Colors.green,    // Text color
    fontWeight: FontWeight.bold, // Makes the text bold
  ),
  textAlign: TextAlign.center, // Centers the text
  maxLines: 2,               // Limits to 2 lines
  overflow: TextOverflow.ellipsis, // Adds "..." if the
text is too long
)
```

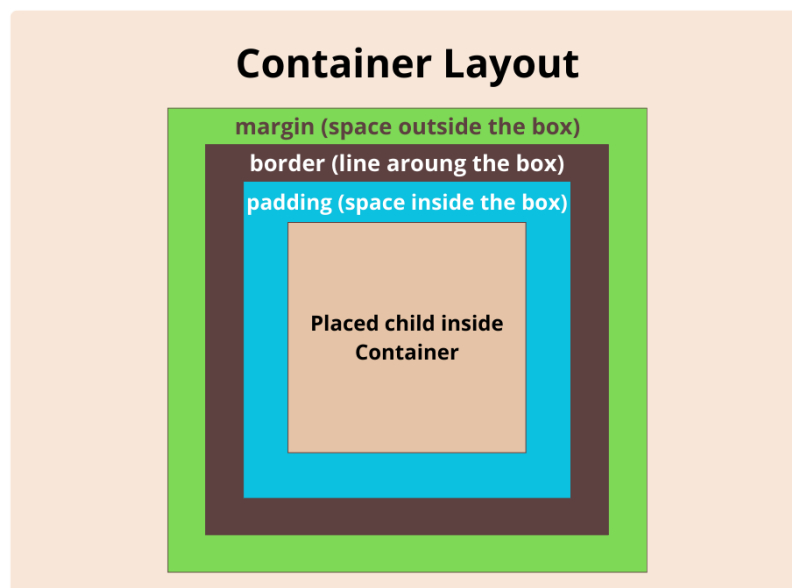
- Explanation :
  - 'Welcome to Flutter!' → The text to display.
  - style → Defines size, color, weight, etc.
  - textAlign: TextAlign.center → Centers the text.
  - maxLines: 2 → Limits text to 2 lines.
  - overflow: TextOverflow.ellipsis → Shows "..." if text is too long.
- d. Why Use the Text Widget?
  - Advantages of using the Text widget:
    - Easy to use for both static & dynamic text.
    - Highly customizable (alignment, wrapping, overflow, etc.).

- Flexible for any layout or UI design.

## Container Widget

- What is the Container Widget?
  - A customizable “box” used to style, position, and layout elements in an app.
  - Often used to wrap other widgets and apply spacing, colors, or decorations.
  - One of the most flexible and commonly used widgets in Flutter.
- Key Features of Container
  - Commonly used properties:

| Property                  | Description                                        | Example                           |
|---------------------------|----------------------------------------------------|-----------------------------------|
| <b>margin</b>             | Adds space outside the container.                  | margin:<br>EdgeInsets.all(10)     |
| <b>padding</b>            | Adds space inside the container.                   | padding:<br>EdgeInsets.all(15)    |
| <b>width &amp; height</b> | Sets the size of the container.                    | width: 200, height: 100           |
| <b>color</b>              | Sets the background color.                         | color: Colors.blue                |
| <b>decoration</b>         | Adds styling like borders, gradients, and shadows. | decoration:<br>BoxDecoration(...) |



- Customizing Container with BoxDecoration
  - Additional styling options with BoxDecoration:
    - color → Sets the background color.
    - border → Adds a border around the container.
    - borderRadius → Rounds the corners of the container.
    - boxShadow → Adds a shadow effect.
    - gradient → Creates a gradient background.
- Example: Styled Container in Flutter
  - Code Example:

```
Container(
  width: 200, // Width of the container
```